Newton-Raphson Root Solver in MATLAB

Numerical Methods Portfolio Project

Author

Ehsan Shakil Ahmad

Developed Using

MATLAB Online

Project Date

June 2026

Project Overview

This project presents the development of a MATLAB program that implements the Newton-Raphson Method for solving nonlinear equations. The program calculates the root of a given function through iterative approximation, displays the convergence process, and visualizes the result graphically. The project demonstrates the application of numerical methods and MATLAB programming in engineering computation.

Keywords: Numerical Methods, MATLAB, Root Finding, Newton-Raphson Method, Derivative-Based Root Finding, Engineering Computation

TABLE OF CONTENTS

1 OBJECTIVES

- To understand the theory behind the Newton-Raphson Method.
- To develop a MATLAB program for solving nonlinear equations.
- To implement convergence monitoring and error handling.
- To visualize the computed root using graphical representation.
- To investigate the effect of different initial guesses on convergence behavior.
- To strengthen MATLAB programming and numerical analysis skills through practical implementation.

2 THEORY

The Newton-Raphson Method starts from an initial guess $x_0$and generates improved approximations using:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

For the function:

$$f(x) = x^3 - x - 2$$

the derivative is:

$$f'(x) = 3x^2 - 1$$

Iterations continue until:

$$| x_{n+1} - x_n | < \text{tol}$$

The method exhibits quadratic convergence near the root, making it significantly faster than interval-based methods.

3 NOMENCLATURE TABLE

Symbol	Description
x_0	Initial guess
x_n	Current approximation
x_{n+1}	Next approximation
f(x)	Function
f'(x)	Derivative
tol	Tolerance
iter	Iteration counter

4 CODE IMPROVEMENT

Several improvements were incorporated into the MATLAB implementation to make the Newton-Raphson Method more interactive, reliable, and suitable for engineering computation. Instead of using a fixed starting value inside the program, the code asks the user to enter an initial guess. This makes the solver more flexible because different starting values can be tested without modifying the main code.

The nonlinear function and its derivative were defined separately using anonymous functions. This improves readability and makes the program easier to modify for other nonlinear equations. The derivative function is especially important because the Newton-Raphson Method is a derivative-based root-finding technique.

A derivative-checking condition was also included before each iteration. If the derivative becomes too small, the program stops execution and displays an error message. This prevents division-by-zero or near-division-by-zero problems, which are common failure cases in the Newton-Raphson Method.

The program also displays the approximation obtained at each iteration in a convergence table. This allows the user to observe how quickly the solution approaches the actual root. Finally, graphical visualization was added by plotting the function and marking the computed root on the graph. This confirms that the numerical result corresponds to the point where the curve crosses the x-axis.

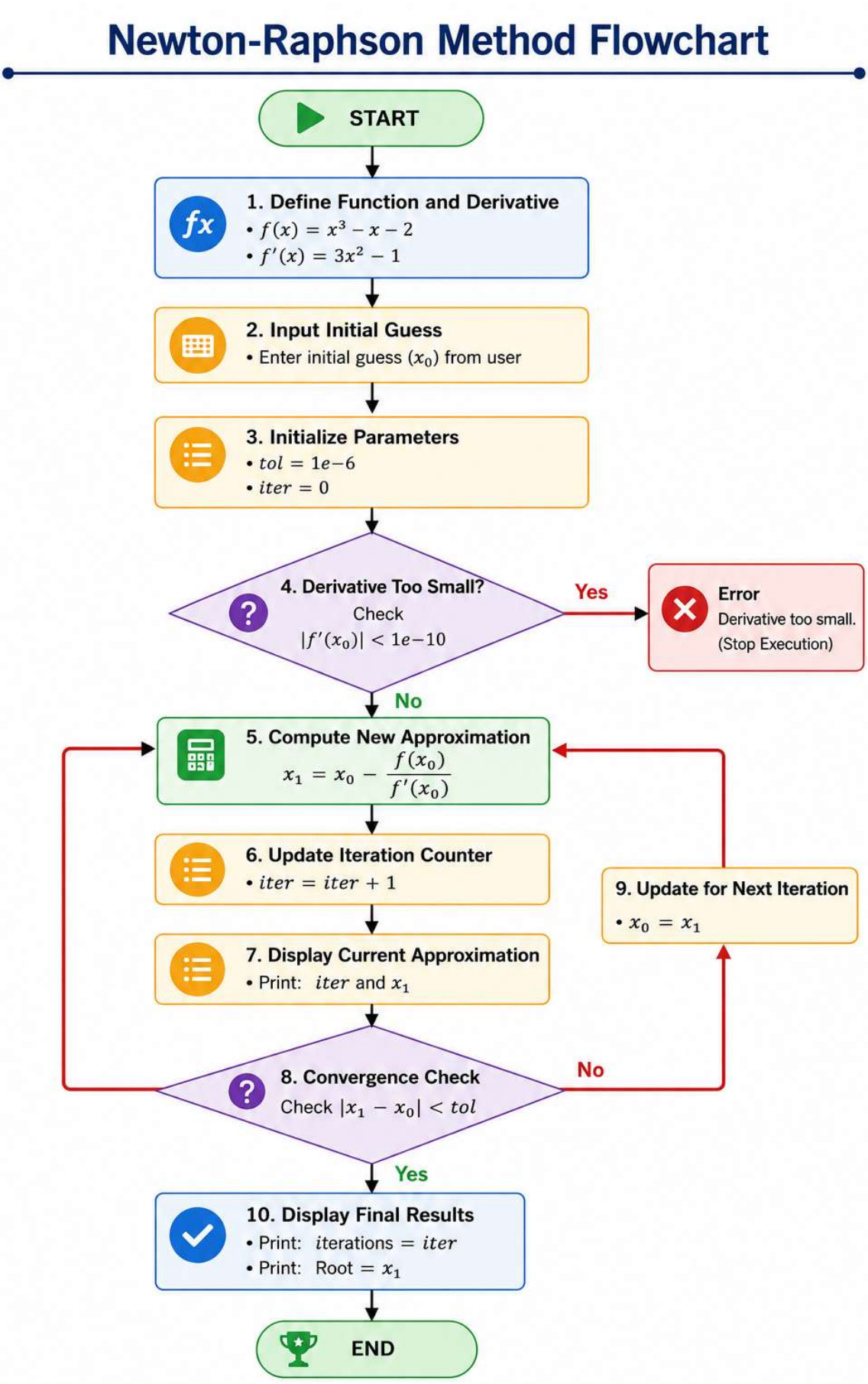
4.1 COMPUTATIONAL EFFICIENCY

The Newton-Raphson Method is computationally efficient because it usually converges very rapidly when the initial guess is close to the actual root. Unlike the Bisection Method, which reduces the interval by half in every iteration, Newton-Raphson uses both the function value and its derivative to move directly toward the root.

For the nonlinear equation ($f(x)=x^3-x-2$), the method converged to the root ($x=1.521380$) in only 3 iterations using the initial guess ($x_0=1.5$). This demonstrates the fast convergence behavior of the method.

However, the efficiency of the Newton-Raphson Method depends strongly on the initial guess and the derivative of the function. If the initial guess is poor, or if the derivative becomes zero or very small, the method may fail, diverge, or converge slowly. Therefore, the derivative-checking condition included in the code improves both reliability and computational safety.

5 FLOWCHART


Newton-Raphson Method Flowchart
START
1. Define Function and Derivative
• $f(x) = x^3 - x - 2$
• $f'(x) = 3x^2 - 1$
2. Input Initial Guess
• Enter initial guess (x_0) from user
3. Initialize Parameters
• $tol = 1e{-}6$
• $iter = 0$
4. Derivative Too Small?
Check
$|f'(x_0)| < 1e{-}10$
Yes
Error
Derivative too small.
(Stop Execution)
No
5. Compute New Approximation
$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$
6. Update Iteration Counter
• $iter = iter + 1$
7. Display Current Approximation
• Print: $iter$ and x_1
8. Convergence Check
Check $|x_1 - x_0| < tol$
No
9. Update for Next Iteration
• $x_0 = x_1$
Yes
10. Display Final Results
• Print: iterations = $iter$
• Print: Root = x_1
END

6 NEWTON RAPHSON CODE

```
% Define the nonlinear function and its derivative
f = @(x) x.^3 - x - 2;
df = @(x) 3*x.^2 - 1;

% Asks the user to Input Initial Guess
x0 = input('Enter initial guess: ');

% Tolerance
tol = 1e-6;

% Iteration Counter
iter = 0;

% Table
fprintf('Iteration\t x\n')

while true

    % This prevents division-by-zero issues.
    if abs(df(x0)) < 1e-10
        error('Derivative too small.')
    end

    x1 = x0 - f(x0)/df(x0);

    iter = iter + 1;

    fprintf('%d\t\t %.8f\n',iter,x1)

    if abs(x1-x0) < tol
        break;
    end

    x0 = x1;

end

% Display Iterations and Root Value
fprintf('iterations = %d\n', iter)
fprintf('Root = %.6f\n', x1)

% Plotting
x = 0:0.1:3;

plot(x,f(x),'r',LineWidth=2)
grid on
hold on

% Mark computed root on graph
plot(x1,f(x1),'bo',Markersize=10)

% Add graph labels and title
xlabel('x')
ylabel('f(x)')
title('Newton Raphson Root Value')
```

7 OUTPUT

7.1 INITIAL VALUES

Parameter	Value
Initial Guess (x_0)	1.500000
Tolerance	1×10^{-6}

7.2 CONVERGENCE TABLE

Table 1 Convergence history of the Newton-Raphson Method for f(x)

Iteration	x_n	x_{n+1}
1	1.50000000	1.52173913
2	1.52173913	1.52137981
3	1.52137981	1.52137971

7.3 GRAPH

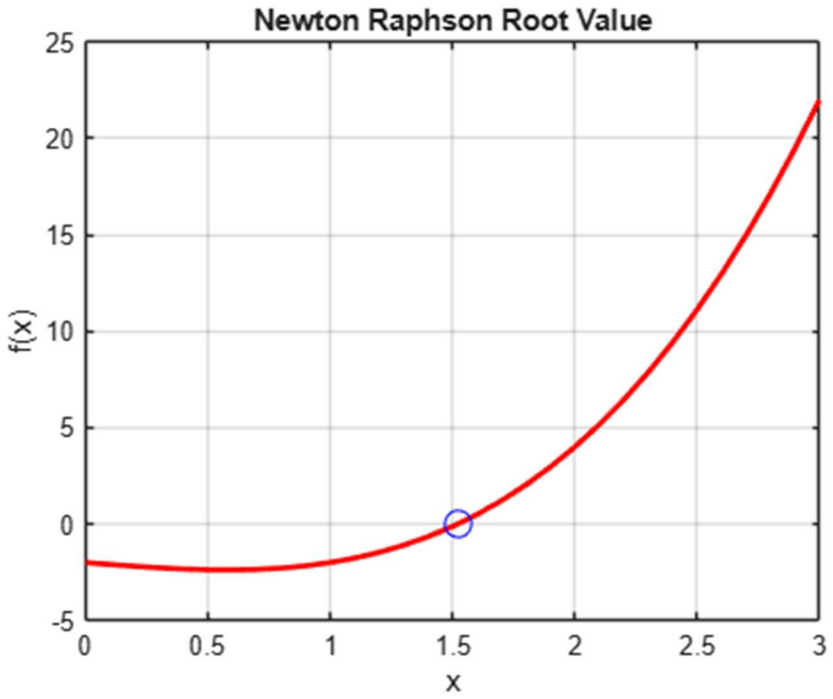


Figure 1 Graph of f(x) showing the root obtained using the Newton-Raphson Method.

8 ROBUSTNESS TESTING

To examine the reliability of the Newton-Raphson Method, the program was tested using different initial guesses for the same nonlinear equation:

$$f(x) = x^3 - x - 2$$

The purpose of this testing was to observe how the method behaves when the starting value is close to the root, far from the root, or near a critical point where the derivative becomes very small.

Case	Initial Guess	Iterations	Root	Observation
1	1.5	3	1.521380	Fast and stable convergence
2	10	9	1.521380	Converged successfully, but required more iterations
3	0	31	1.521380	Large jumps occurred before convergence
4	0.57735	50	1.521380	Very unstable behavior due to small derivative
5	-10	37	1.521380	Poor initial guess caused delayed convergence

The results show that the Newton-Raphson Method is highly efficient when the initial guess is close to the actual root. For example, using (x_0=1.5), the method converged in only 3 iterations. However, when the initial guess was farther from the root, such as (x_0=10), the method still converged but required 9 iterations.

More unstable behavior was observed for (x_0=0), (x_0=0.57735), and (x_0=-10). These cases produced large jumps before eventually converging to the same root. The case (x_0=0.57735) was especially significant because it is close to the point where the derivative of the function becomes zero. Since

$$f'(x) = 3x^2 - 1$$

the derivative becomes zero near

$$x_0 = 0.57735$$

When the derivative is very small, the Newton-Raphson formula produces very large step sizes. This explains the sudden large values observed during the iterations.

Overall, the robustness testing shows that the Newton-Raphson Method can converge very quickly, but its performance strongly depends on the choice of initial guess. A good initial guess leads to fast convergence, while a poor initial guess can cause instability, large jumps, or a much higher number of iterations. Therefore, the derivative-checking condition included in the code improves the safety and reliability of the method.

9 COMPARISON WITH BISECTION METHOD

Both the Bisection Method and the Newton-Raphson Method were implemented in MATLAB to solve the nonlinear equation

$$f(x) = x^3 - x - 2$$

The Bisection Method is an interval-based root-finding technique that guarantees convergence when a valid interval containing a sign change is provided. In contrast, the Newton-Raphson Method uses the derivative of the function and an initial guess to iteratively approach the root.

For the same equation and convergence tolerance of 1×10^{-6}, the Bisection Method required 20 iterations to obtain the root (x=1.521380), whereas the Newton-Raphson Method achieved the same result in only 3 iterations when using an initial guess of (x_0=1.5).

Feature	**Bisection Method**	**Newton-Raphson Method**
Root Obtained	1.521380	1.521380
Tolerance	1×10^{-6}	1×10^{-6}
Iterations Required	20	3
Uses Derivative	No	Yes
Requires Initial Interval	Yes	No
Requires Initial Guess	No	Yes
Convergence Rate	Linear	Quadratic
Guaranteed Convergence	Yes (if sign change exists)	Not always
Computational Efficiency	Moderate	High

For this problem, the Newton-Raphson Method achieved an 85% reduction in iteration count compared to the Bisection Method.

The comparison clearly demonstrates the superior convergence speed of the Newton-Raphson Method. However, the robustness testing performed in this project also revealed that Newton-Raphson is more sensitive to the choice of initial guess. Poor starting values can lead to instability, large oscillations, or slow convergence. The Bisection Method, while slower, remains more reliable because it systematically reduces the search interval and guarantees convergence when a sign change is present.

Therefore, the choice between the two methods depends on the problem requirements. When reliability is the primary concern, the Bisection Method is often preferred. When computational speed is important and a suitable initial guess is available, the Newton-Raphson Method is generally the better choice.

10 RESULTS

The MATLAB implementation successfully determined the root of the nonlinear equation

$$f(x) = x^3 - x - 2$$

using the Newton-Raphson Method. Starting from an initial guess of

$$x_0 = 1.5$$

the algorithm converged to the root

$$\mathrm{x} = 1.521380$$

within only 3 iterations while satisfying the convergence tolerance of

$$1 \times 10^{-6}$$

The convergence table demonstrated the rapid reduction in approximation error, confirming the quadratic convergence characteristics of the Newton-Raphson Method. The generated graph further verified that the computed root corresponds to the point where the function intersects the x-axis. Figure 1 visually confirms that the computed root corresponds to the x-intercept of the function, providing graphical verification of the numerical result.

Compared with the previously implemented Bisection Method solver, the Newton-Raphson Method achieved the same root value while reducing the number of required iterations from 20 to 3, demonstrating a substantial improvement in computational efficiency.

The robustness testing showed that the method remained capable of converging to the same root for a wide range of initial guesses. However, the number of iterations varied significantly depending on the starting value. Initial guesses close to the root produced rapid convergence, whereas guesses near points where the derivative approaches zero resulted in large oscillations and substantially slower convergence.

Quantity	Value
Function	$x^3 - x - 2$
Initial Guess	1.500000
Computed Root	1.521380
Tolerance	1×10^{-6}
Iterations	3
Method	Newton-Raphson

The results confirm both the accuracy and efficiency of the Newton-Raphson Method for solving nonlinear equations when an appropriate initial guess is selected. Such root-finding techniques are widely used in engineering applications including fluid mechanics, thermodynamics, heat transfer, structural analysis, and optimization problems.

11 CONCLUSION

A MATLAB-based Newton-Raphson root solver was successfully developed, implemented, and tested. The program utilized user-defined initial guesses, derivative evaluation, convergence monitoring, error handling, and graphical visualization to accurately determine the root of a nonlinear equation.

For the test function $f(x) = x^3 - x - 2$, the solver converged to the root $\mathrm{x} = 1.521380$ in only 3 iterations when starting from an initial guess of $x_0 = 1.5$. This demonstrates the high computational efficiency of the Newton-Raphson Method compared to interval-based techniques such as the Bisection Method.

The robustness testing highlighted the importance of selecting a suitable initial guess. Although convergence was achieved for all tested cases, poor starting values produced larger iteration counts and unstable intermediate approximations. The derivative-checking mechanism proved valuable in preventing numerical issues associated with extremely small derivative values.

A direct comparison with the Bisection Method showed that Newton-Raphson converges significantly faster for the same nonlinear equation. Although the method is more sensitive to the initial guess, its superior convergence rate makes it highly attractive for engineering computations where efficiency is important.

Overall, this project strengthened understanding of numerical root-finding algorithms, convergence behavior, derivative-based methods, and MATLAB programming for engineering applications. Future work may include implementing and comparing other numerical methods such as the Secant Method, False Position Method, Fixed Point Iteration, and higher-dimensional Newton-based solvers.